# Predicting 28-Day Mortality in Hospitalized Heart Failure Patients Using a Hyperparameter-Optimized Random Forest Classifier

**Rohit Kumar**

## Abstract

Heart failure is a major cardiovascular condition associated with hospitalization, morbidity, and mortality. Early identification of patients at high risk of short-term mortality can support clinical risk stratification and healthcare resource planning. However, mortality prediction from electronic health records (EHRs) presents several machine learning challenges, including severe class imbalance, heterogeneous clinical variables, missing observations, and nonlinear relationships between patient characteristics and clinical outcomes.

This paper presents a machine learning framework for predicting 28-day mortality among hospitalized heart failure patients using a Random Forest classifier. The study uses a publicly available EHR dataset containing 2,008 hospitalized heart failure patients and 167 initial variables. A systematic preprocessing pipeline was implemented, including removal of variables with more than 90% missing observations, removal of an index column, categorical missing-value replacement, numerical mean imputation, and one-hot encoding of categorical variables. The processed dataset was divided into training and testing subsets using a 70:30 split with a fixed random state of 100. Because only 1.84% of observations corresponded to 28-day mortality, Synthetic Minority Oversampling Technique (SMOTE) was applied exclusively to the training data, producing a balanced training set of 2,766 observations.

A baseline Random Forest classifier achieved a minority-class recall of 0.53 and an F1-score of 0.70 on the held-out test set. To improve minority-class detection, RandomizedSearchCV with five-fold cross-validation was used to optimize the number of trees and maximum tree depth. The optimal configuration was a Random Forest with 150 estimators and maximum depth of 2. The optimized model achieved a mortality-class precision of 1.00, recall of 0.93, and F1-score of 0.97 on the 603-patient test set, with an overall accuracy of 1.00 when rounded to two decimal places. The results demonstrate that model optimization substantially improved detection of the minority mortality class while maintaining high overall classification performance.

**Index Terms—** Heart failure, electronic health records, machine learning, Random Forest, SMOTE, class imbalance, mortality prediction, hyperparameter optimization, clinical prediction.

---

# I. INTRODUCTION

Heart failure (HF) is a complex cardiovascular syndrome associated with substantial morbidity, recurrent hospitalization, and mortality. The clinical condition is heterogeneous, with patient outcomes influenced by physiological status, cardiac function, comorbidities, laboratory measurements, and treatment-related variables.

The increasing availability of electronic health records has created opportunities for computational models to assist with clinical risk prediction. EHR datasets contain large numbers of heterogeneous variables, including demographic information, vital signs, laboratory measurements, echocardiographic findings, clinical assessments, and medication information. Machine learning algorithms can exploit these variables to identify nonlinear relationships that may be difficult to capture using conventional statistical models.

A particularly important challenge in mortality prediction is class imbalance. In highly imbalanced datasets, a classifier can obtain apparently high accuracy by primarily predicting the majority class while failing to identify patients belonging to the minority mortality class. This problem is especially important in clinical prediction because false-negative predictions may correspond to high-risk patients whose mortality is not identified.

The present study addresses this problem by combining preprocessing, SMOTE-based minority-class oversampling, Random Forest classification, and hyperparameter optimization.

The major contributions of this study are:

1. Development of an end-to-end EHR preprocessing and classification pipeline for 28-day mortality prediction.
2. Quantification of severe class imbalance in the target variable.
3. Application of SMOTE exclusively to the training dataset.
4. Development of a baseline Random Forest classifier.
5. Optimization of Random Forest hyperparameters using RandomizedSearchCV with five-fold cross-validation.
6. Quantitative comparison between the baseline and optimized Random Forest models.
7. Analysis of the resulting feature importance for the optimized model.

---

# II. DATASET

## A. Data Source

The study uses a publicly available heart failure EHR dataset containing records collected from Zigong Fourth People's Hospital in Sichuan Province, China.

The original dataset contains 2,008 patient records and 167 variables. The underlying clinical study collected information from hospitalized heart failure patients admitted between December 2016 and June 2019.

The dataset includes clinical measurements recorded during hospitalization and follow-up information concerning patient outcomes.

The original dataset was designed to support research involving hospitalized heart failure patients and includes information from routine electronic healthcare records.

---

## B. Clinical Variables

The dataset contains heterogeneous clinical variables.

### Physiological Variables

Examples include:

- Body temperature
- Pulse
- Respiration rate
- Systolic blood pressure
- Diastolic blood pressure
- Mean arterial pressure
- Weight
- Height
- Body mass index

### Clinical Assessment Variables

The dataset contains:

- Type of heart failure
- NYHA classification
- Killip grade
- Glasgow Coma Scale (GCS)

### Echocardiographic Variables

Representative cardiac measurements include:

- Left ventricular ejection fraction
- Left ventricular end-diastolic diameter
- Mitral valve peak E-wave velocity
- Mitral valve peak A-wave velocity
- E/A ratio
- Tricuspid regurgitation velocity
- Tricuspid regurgitation pressure

### Medication Variables

The dataset includes medication-related information such as:

- Furosemide
- Torsemide
- Spironolactone
- Dobutamine
- Digoxin
- Milrinone
- Nitroglycerin
- Isosorbide mononitrate

The original dataset contains additional medication information and clinical variables.

---

# III. PROBLEM FORMULATION

The objective is to predict whether a hospitalized heart failure patient experiences mortality within 28 days.

Let the feature vector for patient $i$ be

$$\mathbf{x}_i = [x_{i1}, x_{i2}, ..., x_{ip}]$$

where $p$ represents the number of processed input features.

The binary target is

$$y_i \in \{0, 1\}$$

where

$$y_i = \begin{cases} 0, & \text{patient does not die within 28 days } 1, \ \text{patient dies within 28 days} \end{cases}$$

The machine learning objective is to learn a classifier

$$f(\mathbf{x}) \to y$$

that can accurately classify previously unseen patients.

The target variable used in the implementation is:

`death.within.28.days`

---

# IV. DATA PREPROCESSING

The preprocessing pipeline was implemented using Python and Pandas.

The original dataset contained 2,008 observations and 167 columns.

## A. Removal of Highly Missing Variables

Variables containing more than 90% missing observations were removed.

The implementation used the following criterion:

$$\frac{\text{Number of missing observations in feature } j}{\text{Total observations}} > 0.90$$

If this condition was satisfied, the corresponding feature was removed.

After this operation, the dataset contained 159 columns.

This step reduces the dimensionality of the dataset and avoids retaining variables with extremely limited observed information.

---

## B. Removal of Index Variable

The `Unnamed: 0` column was removed because it represents an index-like variable rather than a clinical predictor.

The implementation explicitly removed this variable before subsequent modeling.

After removal, 158 columns remained before categorical encoding.

---

## C. Categorical Missing-Value Imputation

For variables with object/string data types, missing observations were replaced by a blank string.

The implementation performed:

$$x_{ij} = \text{blank} \quad \text{if } x_{ij} \text{ is missing and categorical}$$

This was applied to all object-type variables.

---

## D. Numerical Missing-Value Imputation

Missing values in numerical variables were replaced with the corresponding column mean.

For feature $j$, the imputed value is

$$x_{ij} = \bar{x}_j$$

where

$$\bar{x}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} x_{ij}$$

and $n_j$ represents the number of observed values for feature $j$.

The implementation explicitly applied mean imputation to numerical variables.

---

## E. Categorical Encoding

Categorical variables were converted into numerical representations using one-hot encoding.

The implementation used:

```
pd.get_dummies(..., drop_first=True)
```

for object-type variables.

The `drop_first=True` option removes one category from each categorical variable and provides a reference category.

---

# V. EXPLORATORY ANALYSIS

Exploratory data analysis was performed before model development to understand the structure and distribution of the data.

The analysis examined:

- Variable distributions
- Missing-value patterns
- Target distribution
- Relationships between clinical variables
- Categorical variables
- Numerical variables

The target distribution revealed severe class imbalance.

The proportion of observations belonging to the two classes was:

$$P(y = 0) = 0.981574$$

and

$$P(y = 1) = 0.018426$$

Therefore, approximately 98.16% of observations belonged to the non-mortality class and approximately 1.84% belonged to the mortality class.

This imbalance creates a significant challenge for mortality prediction.

---

# VI. TRAIN-TEST PARTITIONING

The processed dataset was divided into training and testing subsets using `train_test_split`.

The implementation used:

- Training proportion: 70%
- Testing proportion: 30%
- Random state: 100

The target variable was separated from the feature matrix before splitting.

The resulting test set contained 603 patients, including:

- 588 non-mortality cases
- 15 mortality cases

The test set was retained without SMOTE and was used to evaluate generalization performance.

---

# VII. CLASS IMBALANCE HANDLING USING SMOTE

## A. Motivation

The original mortality class represented only approximately 1.84% of the dataset.

If the classifier were trained directly on the original distribution, it could become biased toward the majority class.

For example, a classifier predicting nearly every patient as a survivor could still obtain high accuracy while failing to detect mortality events.

Therefore, the training dataset was balanced using SMOTE.

---

## B. SMOTE Formulation

Synthetic Minority Oversampling Technique generates synthetic minority-class observations using neighboring minority samples.

For a minority observation $x_i$ and one of its nearest minority neighbors $x_{nn}$, a synthetic observation can be expressed as:

$$x_{new} = x_i + \lambda(x_{nn} - x_i)$$

where

$$\lambda \in [0, 1]$$

is a randomly generated interpolation factor.

The implementation used the standard `SMOTE()` configuration and applied:

$$X_{train}^{SMOTE}, Y_{train}^{SMOTE} = SMOTE.fit\_resample(X_{train}, Y_{train})$$

The notebook confirms that SMOTE was applied after the train-test split, meaning the test data remained untouched.

The resulting training dataset contained 2,766 observations:

$$N_{class0} = 1383$$

$$N_{class1} = 1383$$

Thus, the training data became exactly balanced.

---

# VIII. RANDOM FOREST CLASSIFIER

## A. Ensemble Structure

Random Forest is an ensemble learning algorithm that constructs multiple decision trees and aggregates their predictions.

For $B$ decision trees,

$$\hat{y} = mode\{T_1(x), T_2(x), ..., T_B(x)\}$$

where $T_b(x)$ represents the prediction generated by tree $b$.

The ensemble reduces the dependence on an individual decision tree and can capture nonlinear relationships between predictors and outcomes.

---

## B. Gini Impurity

For a node containing two classes, the Gini impurity is:

$$Gini = 1 - (p_0^2 + p_1^2)$$

where $p_0$ and $p_1$ represent the class proportions within the node.

A decision-tree split attempts to produce child nodes with lower impurity.

---

## C. Baseline Model

The initial model was created using:

$$\texttt{RandomForestClassifier()}$$

with the default Random Forest configuration.

The model was trained using the SMOTE-balanced training dataset.

---

# IX. HYPERPARAMETER OPTIMIZATION

The baseline model was subsequently optimized using RandomizedSearchCV.

The search procedure used:

- 100 randomized iterations
- 5-fold cross-validation
- Parallel execution using `n_jobs=-1`
- `n_estimators` search range: 100 and 150
- `max_depth` search range: 1 and 2

The exact parameter distributions in the notebook were:

$$n\_estimators \in \{100, 150\}$$

$$max\_depth \in \{1, 2\}$$

The RandomizedSearchCV configuration was explicitly recorded in the notebook.

The optimized model was:

$$\boxed{RandomForestClassifier(n\_estimators = 150,\ max\_depth = 2)}$$

The notebook reports this as the selected `best_estimator_`.

---

# X. EXPERIMENTAL EVALUATION

The models were evaluated using:

- Precision
- Recall
- F1-score
- Accuracy
- Confusion matrix

The analysis emphasizes the mortality class because it represents the clinically important minority class.

For binary classification:

## Precision

$$Precision = \frac{TP}{TP + FP}$$

## Recall

$$Recall = \frac{TP}{TP + FN}$$

## F1-score

$$F1 = 2\frac{Precision \times Recall}{Precision + Recall}$$

## Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

---

# XI. RESULTS

## A. Baseline Random Forest

The baseline Random Forest achieved the following performance on the test set:

### TABLE I

### BASELINE RANDOM FOREST PERFORMANCE

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| 0 — Non-mortality | 0.99 | 1.00 | 0.99 | 588 |
| 1 — Mortality | 1.00 | 0.53 | 0.70 | 15 |
| **Overall Accuracy** | | | **0.99** | **603** |

The baseline model correctly identified the majority of non-mortality cases but detected only approximately 53% of the mortality cases.

This result demonstrates why accuracy alone is insufficient for this problem.

Although the baseline achieved approximately 99% accuracy, the mortality-class recall was only 0.53.

In practical terms, the baseline model failed to identify approximately half of the mortality cases in the held-out test set.

---

# XII. OPTIMIZED RANDOM FOREST RESULTS

The optimized model used:

$$n\_estimators = 150$$

and

$$max\_depth = 2$$

The resulting test performance was:

**TABLE II**

**OPTIMIZED RANDOM FOREST PERFORMANCE**

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| 0 — Non-mortality | 1.00 | 1.00 | 1.00 | 588 |
| 1 — Mortality | **1.00** | **0.93** | **0.97** | 15 |
| **Overall Accuracy** | | | **1.00** | **603** |
| Macro Average | 1.00 | 0.97 | 0.98 | 603 |
| Weighted Average | 1.00 | 1.00 | 1.00 | 603 |

The notebook reports the optimized model's overall accuracy as 1.00 after rounding, with mortality precision of 1.00, mortality recall of 0.93, and mortality F1-score of 0.97.

The improvement in mortality recall is particularly important.

The baseline mortality recall was:

$$0.53$$

whereas the optimized model achieved:

$$0.93$$

This represents an improvement of approximately:

$$0.93 - 0.53 = 0.40$$

or approximately **40 percentage points** in reported recall.

---

# XIII. CONFUSION-MATRIX ANALYSIS

The test set contained 603 observations:

$$588 \text{ non-mortality}$$

and

$$15 \text{ mortality}$$

For the optimized classifier, the reported precision and recall correspond to approximately:

$$TN = 588$$

$$FP = 0$$

$$FN = 1$$

$$TP = 14$$

Therefore, the confusion matrix can be represented as:

### TABLE III

### CONFUSION MATRIX OF OPTIMIZED RANDOM FOREST

| Actual / Predicted | Non-Mortality | Mortality |
|---|---|---|
| Non-Mortality | 588 | 0 |
| Mortality | 1 | 14 |

The optimized model therefore identified 14 of the 15 mortality cases in the held-out test set.

The corresponding mortality recall is:

$$Recall = \frac{14}{14+1} = 0.9333$$

which is consistent with the reported 0.93 value.

The corresponding accuracy is:

$$Accuracy = \frac{588+14}{603} = 0.9983$$

or approximately:

$$99.83\%$$

The classification report rounds this value to 1.00.

---

# XIV. COMPARATIVE ANALYSIS

### TABLE IV

### BASELINE VS. OPTIMIZED RANDOM FOREST

| Metric | Baseline RF | Optimized RF |
|---|---|---|
| Accuracy | 0.99 | 1.00 |

| Metric | Baseline RF | Optimized RF |
|---|---|---|
| Mortality Precision | 1.00 | 1.00 |
| Mortality Recall | 0.53 | **0.93** |
| Mortality F1-score | 0.70 | **0.97** |
| Test samples | 603 | 603 |

The primary improvement occurred in minority-class detection.

The mortality recall increased from 0.53 to 0.93, while the mortality F1-score increased from 0.70 to 0.97.

This demonstrates that hyperparameter optimization substantially improved the ability of the model to identify mortality cases.

---

# XV. TRAINING PERFORMANCE

The SMOTE-balanced training dataset contained 2,766 observations, with 1,383 observations in each class.

The baseline Random Forest achieved:

- Precision = 1.00
- Recall = 1.00
- F1-score = 1.00
- Accuracy = 1.00

on the SMOTE-balanced training data.

The optimized model also achieved:

- Precision = 1.00
- Recall = 1.00
- F1-score = 1.00
- Accuracy = 1.00

on the balanced training dataset.

The perfect training performance should not be interpreted as proof of perfect generalization. The held-out test performance is the more relevant estimate of behavior on unseen observations.

---

# XVI. FEATURE IMPORTANCE

Feature importance was calculated using the fitted optimized Random Forest.

For each feature $j$, the model provides an impurity-based importance value:

$$FI_j$$

The implementation created a feature-importance DataFrame using the model's feature names and `feature_importances_` values and then sorted the features in descending order to obtain the top ten predictors.

### FIG. 1. Top 10 feature importance values generated from the optimized Random Forest.

**[INSERT THE TOP-10 FEATURE-IMPORTANCE PLOT GENERATED BY THE NOTEBOOK HERE]**

The feature-importance analysis provides an additional interpretation layer by identifying variables that contributed most strongly to the tree-based decision process.

However, impurity-based feature importance should not be interpreted as causal evidence. A high feature-importance value indicates predictive contribution within the trained model and does not establish that modifying that variable would directly modify mortality risk.

---

# XVII. DISCUSSION

The experimental results demonstrate an important distinction between overall classification accuracy and minority-class detection.

The dataset contains a highly imbalanced target distribution, with only approximately 1.84% of observations corresponding to mortality.

Under this distribution, a model can achieve high accuracy without adequately identifying mortality cases.

This behavior is visible in the baseline Random Forest. The baseline model obtained approximately 99% test accuracy but achieved only 0.53 recall for the mortality class.

Following hyperparameter optimization, mortality recall increased to 0.93 and F1-score increased to 0.97.

This suggests that the optimized configuration provided substantially better separation of the minority class under the selected experimental setup.

An important methodological feature is that SMOTE was applied after the train-test split. Therefore, the synthetic observations were generated only from the training data, while the original test set was retained for evaluation.

This design avoids directly oversampling the held-out test data.

---

# XVIII. TECHNICAL INTERPRETATION OF THE OPTIMIZED MODEL

The optimal model identified by RandomizedSearchCV was relatively shallow:

$$max\_depth = 2$$

with:

$$n\_estimators = 150$$

The shallow depth limits the complexity of individual trees while the ensemble combines predictions from 150 trees.

The selected model therefore provides an interesting result: the highest-performing configuration in the conducted search did not require deep decision trees.

However, the hyperparameter search space was limited to only two parameters:

$$n\_estimators \in \{100, 150\}$$

and

$$max\_depth \in \{1, 2\}$$

Therefore, the selected configuration should be interpreted as the best configuration **within the tested search space**, rather than a globally optimal Random Forest configuration.

---

# XIX. LIMITATIONS

Several limitations should be considered when interpreting the results.

## A. Severe Class Imbalance

Only 1.84% of the original observations belonged to the mortality class.

Although SMOTE improved representation of the minority class during training, synthetic observations cannot replace additional real clinical observations.

---

## B. Single-Center Dataset

The dataset originates from a single hospital.

Therefore, the model may learn hospital-specific patterns that do not generalize to other healthcare institutions.

---

## C. Retrospective Data

The data were collected retrospectively from electronic health records.

Retrospective datasets can contain missing observations, documentation inconsistencies, and institution-specific measurement patterns.

---

## D. Limited Temporal Information

The source dataset is aggregated at the hospitalization level and does not provide complete longitudinal measurements during hospitalization.

Medication administration timestamps are also unavailable.

Consequently, the current model does not explicitly model temporal disease progression.

---

## E. Limited Hyperparameter Search

The RandomizedSearchCV experiment searched only:

- `n_estimators`
- `max_depth`

Other Random Forest parameters such as `min_samples_split`, `min_samples_leaf`, `max_features`, class weighting, and bootstrap configuration were not included in the reported search space.

---

## F. No External Validation

The model was evaluated using a held-out test set originating from the same dataset.

External validation using an independent hospital or patient population was not performed.

Therefore, the reported performance should not be interpreted as evidence of clinical readiness.

---

# XX. FUTURE WORK

Future research can extend the proposed framework in several directions.

## A. Expanded Hyperparameter Optimization

A larger search space can be investigated:

$$\Theta = \{n\_estimators, max\_depth, min\_samples\_split, min\_samples\_leaf, max\_features, class\_weight\}$$

This could provide a more comprehensive optimization of the Random Forest classifier.

---

## B. Comparative Machine Learning Models

Future experiments should compare the optimized Random Forest with:

- Logistic Regression
- Decision Trees
- Gradient Boosting
- XGBoost
- LightGBM
- Support Vector Machines
- Neural Networks

Such comparisons would determine whether Random Forest provides a competitive solution for this mortality-prediction problem.

---

## C. Explainable AI

Explainable artificial intelligence methods such as SHAP can be incorporated to quantify how individual clinical variables influence predictions.

For example, SHAP-based analysis could provide both global feature importance and patient-level explanations.

---

### D. Probability Calibration

Future work should evaluate whether predicted probabilities are calibrated.

Potential techniques include:

- Platt scaling
- Isotonic regression
- Calibration curves
- Brier score

This is particularly important if the model is eventually used to generate patient-specific risk probabilities.

---

### E. External Validation

The model should be tested on independent datasets from other hospitals and populations.

A robust evaluation should measure whether discrimination and calibration remain stable under distribution shift.

---

### F. Longitudinal Modeling

Future systems could incorporate time-dependent patient information.

Sequential models such as recurrent neural networks and transformer-based architectures could be investigated when longitudinal EHR data are available.

---

## XXI. CONCLUSION

This study presented a machine learning framework for predicting 28-day mortality among hospitalized heart failure patients using structured electronic health records.

The proposed pipeline combines missing-value processing, high-missingness feature removal, categorical encoding, train-test partitioning, SMOTE-based class balancing, Random Forest classification, and RandomizedSearchCV-based hyperparameter optimization.

The original dataset exhibited substantial class imbalance, with approximately 98.16% non-mortality observations and 1.84% mortality observations. The application of SMOTE produced a balanced training set containing 2,766 observations.

The baseline Random Forest achieved 0.53 recall and 0.70 F1-score for the mortality class on the held-out test set. After hyperparameter optimization, the selected model used 150 trees and a maximum tree depth of 2. The optimized model achieved a mortality recall of 0.93 and F1-score of 0.97, while the reported overall accuracy was 1.00 after rounding.

The most important finding is therefore not simply the high overall accuracy but the substantial improvement in minority-class mortality detection.

The results demonstrate the potential of combining class-imbalance handling with ensemble learning and hyperparameter optimization for structured EHR-based mortality prediction. Nevertheless, the model requires external validation, calibration analysis, broader comparative experiments, and prospective clinical evaluation before it could be considered for real-world clinical decision support.

---

# REFERENCES


[1] Z. Zhang, L. Cao, R. Chen, Y. Zhao, L. Lv, Z. Xu, and P. Xu, "Electronic healthcare records and external outcome data for hospitalized patients with heart failure," *Scientific Data*, vol. 8, no. 1, Art. no. 46, 2021, doi: 10.1038/s41597-021-00835-9.

[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.